

【金融資料探勘】

2021 年度選擇權 IV 數據工程與敘述性統計分析報告

學生姓名： 楊栩函

學號： 411121265

組別： 第一組

負責年度： 2021

分析日期： 2026 / 04 / 29

一、 作業目的

本作業本作業之核心目標在於利用 Python 於 Google Colab 雲端環境中，針對 2021 年台灣選擇權全年度逐筆交易原始資料 (OptionsDaily) 進行自動化清洗與財務分析。透過與 AI 協作開發，解決原始資料中格式混亂與非結構化之干擾，並實作自動化資料工程、隱含波動率 (IV) 計算、流動性雜訊篩選、統計摘要產出

二、 原始資料說明

1. 資料背景

本研究採用之原始數據為 2021 年台灣選擇權市場之每日交易紀錄：

- 資料範疇：涵蓋 2021 年 1 月至 12 月之完整交易日，總計 244 個 CSV 原始檔案。
- 儲存位置：檔案存放於雲端硬碟 HW2/Option_2021 資料夾中。
- 核心欄位：包含成交日期、商品代號、履約價格、到期月份、買賣權別、成交价格（權利金）及成交數量等資訊。

2. 資料清理邏輯

針對原始資料之「髒資料」特性，系統採取以下「暴力清洗」策略以確保處理不中斷：

- 物理性雜訊排除：利用 Python 原生讀取邏輯，物理性過濾掉包含 ---- 之分隔線及空白列，解決 `pd.read_csv` 解析欄位數量不一的報錯問題。
- 智慧錨點定位：因應欄位位移，系統以 C（買權標記）為中心進行相對位移搜尋，確保能精準抓取對應的履約價與成交價。
- 字串去空白處理：對所有文字型欄位執行 `strip()` 處理，確保在比對商品代號與買賣別時，不會因不可見空格導致匹配失敗。
- 強制轉型與過濾：透過 `pd.to_numeric` 進行數據規格化，並嚴格執行「成交量 > 30」之過濾條件，僅保留具備流動性之有效樣本進行後續 BS 模型運算。

三、 資料建構流程

實作過程分為四個主要階段，旨在將原始 Tick 資料轉化為高品質的統計報表：

1. **環境建置與動態掛載：** 首先執行 Google Drive 智慧掛載檢查，確保路徑 /Colab Notebooks/HW2/Option_2021 正確。針對全年度 244 個 CSV 檔案，利用 `os.listdir` 與 `sorted` 建立批次處理清單，確保分析流程具備時間連續性。
2. **暴力清洗與逐行解析：** 針對原始 CSV 檔案中存在的虛線分隔線 (-----) 與非標準欄位間距，捨棄傳統 `read_csv` 解析。改採原生 `split()` 與 `strip()` 技術進行「暴力清洗」，解決了 Expected fields 報錯，並透過「智慧錨點定位」精準捕捉成交價與履約價資訊。
3. **金融篩選與 IV 即時運算：** 導入 Black-Scholes 模型與二分法收斂邏輯。在處理過程中實施雙重過濾：
 - **屬性過濾：** 僅保留「買權 (Call)」資料。
 - **流動性過濾：** 僅保留「成交量 > 30」之交易紀錄，藉此排除市場雜訊。系統於記憶體中 (In-memory) 完成 IV 反推，不產生中間暫存檔以優化雲端執行效能。
4. **統計聚合與報表導出：** 利用 `groupby('Date')` 對全年度資料進行分群，並透過 `describe()` 函數自動生成八項標準敘述性統計指標。最後，將日期格式正規化為 YYYY/M/D 並以 utf-8-sig 編碼導出，確保產出之 PCR_Report_2021.csv 符合專業財務分析需求。

四、 最終索引檔欄位說明

欄位名稱 (Column Name)	資料類型	說明 (Description)	財務意義與用途
Date	Date	交易日期	資料記錄的基準日（已依此進行每日聚合）。
Call_IV_Mean	Float	買權 IV 日均值	當日所有 Call 合約隱含波動率的算術平均數。
Put_IV_Mean	Float	賣權 IV 日均值	當日所有 Put 合約隱含波動率的算術平均數。

Call_IV_Std	Float	買權 IV 標準差	反映市場對「看漲預期」的分歧程度。
Put_IV_Std	Float	賣權 IV 標準差	反映市場對「看跌避險」的分歧程度（數值越高代表恐慌越不對稱）。
Call_Count	Integer	買權有效樣本數	當日經過濾（如成交量 > 30）後的 Call 合約總筆數。
Put_Count	Integer	賣權有效樣本數	當日經過濾後的 Put 合約總筆數。
PCR (Trade Count)	Float	成交筆數 PCR	計算公式為 $\text{Put_Count} / \text{Call_Count}$ 。

五、 成果展示

	A	B	C	D	E	F	G	H	I
1	Date	Call_IV_Mean	Put_IV_Mean	Call_IV_Std	Put_IV_Std	Call_Count	Put_Count	PCR (Trade Count)	
2	2021/1/4	0.061160091	0.11455973	0.04908622	0.0720917	1595	2199	1.378683	
3	2021/1/5	0.039118292	0.083603697	0.04040322	0.0500395	1862	3024	1.62406	
4	2021/1/6	0.061565203	0.05492612	0.05436572	0.0889704	5178	6990	1.349942	
5	2021/1/7	0.107968504	0.19004564	0.04021103	0.0637712	1618	2081	1.286156	
6	2021/1/8	0.117649211	0.190674025	0.04499364	0.078889	1870	2233	1.194118	
7	2021/1/11	0.099286369	0.182155629	0.04742873	0.0740465	1498	1936	1.29239	
8	2021/1/12	0.109760566	0.138155538	0.04075657	0.1005253	2084	2581	1.238484	
9	2021/1/13	0.03832021	0.125393181	0.06840515	0.1001728	3559	5095	1.431582	
10	2021/1/14	0.215684774	0.272081019	0.02181808	0.0856278	751	1335	1.77763	

六、 問題與修正

在處理 2021 年 244 個交易日原始資料的過程中，主要遭遇以下技術挑戰並順利完成修正：

- CSV 格式解析報錯 (Expected Fields Error)：
 - 問題：原始資料中包含大量 ----- 虛線分隔列，且部分行數因空格分佈不均，導致 `pd.read_csv` 出現欄位數量不符的錯誤（如 Expected 15 fields, saw 16）進而中斷程式。

- 修正：捨棄標準解析引擎，改採「逐行物理切割法」。利用 Python 原生 `split()` 與 `strip()` 技術，物理性過濾掉包含橫線的雜訊列，並直接對每個元素進行去空白處理，徹底解決解析異常。
- 欄位位移與資料誤判：
 - 問題：由於標題名稱可能被截斷或帶有隱形空格，導致無法固定透過索引（Index）抓取正確的成交價或履約價，出現 S:0（無標的價數據）的清洗失敗狀況。
 - 修正：開發「智慧錨點定位邏輯」。不再死板指定第幾欄，而是先在該列尋找關鍵字 C（買權），再以此為中心點向前後進行相對偏移抓取，大幅提升了對非標準格式資料的抓取成功率。
- Python 縮進與編碼衝突（Indentation & Encoding）：
 - 問題：在合併程式碼時出現 `IndentationError`（縮進錯誤），且部分檔案因編碼（Big5 vs UTF-8）差異出現亂碼。
 - 修正：重新對齊代碼區塊層級，並導入 `chardet` 套件進行全自動編碼偵測，確保 244 個檔案在批次讀取時均能正常解析中文字元。
- 數據雜訊干擾（Liquidity Noise）：
 - 問題：初期計算出的 IV 包含許多極端值，經查發現是低流動性的小額交易導致價格偏離。
 - 修正：在清洗階段加入「成交量 > 30」的硬性過濾門檻，並僅針對 TX0 買權進行運算，使最終統計結果更符合市場真實波動情況。

七、 AI Prompt 實作與優化紀錄摘要

本作業透過 AI 協作，將複雜的數據清洗與金融計算流程自動化，優化歷程如下：

- 任務目標：自動化解析 2021 全年度 OptionsDaily 原始檔，篩選高流動性買權並產出 8 項標準敘述性統計報表。
- 最佳指令（Master Prompt）：

「請撰寫 Python 程式批次處理 HW2/Option_2021 資料夾下的所有 CSV。使用逐行讀取與 `strip()` 徹底清洗資料，過濾掉包含橫線的無效列。以關鍵字 'C' 作為錨點定位抓取履約價與成交價。

篩選條件為：買權 (C) 且成交量 > 30。整合 Black-Scholes 與二分法計算 IV。最後請不要儲存中間清洗檔，直接在記憶體中運算並產出每日統計摘要表，並以 2021/1/4 格式顯示日期。」

- **優化成效：**

- 高容錯性：成功開發出能應對 244 個格式不一檔案的「暴力清洗」腳本。
- 計算效率：透過 In-memory（記憶體即時處理）技術，取代了原本「先清洗存檔、再讀取計算」的低效流程，顯著縮短全年度數據處理時間。
- 格式標準化：強制要求輸出格式對齊 Excel 範本，確保產出的 PCR_Report_2021.csv 具備專業報表品質。

八、學習反思

在本次 2021 年度選擇權數據探勘的實作過程中，我深刻體會到將財務專業知識轉化為自動化程式邏輯的巨大挑戰。原先以為寫出 Black-Scholes 公式是整個專案的難點，但在實際操作 OptionsDaily 原始資料後發現，處理雜亂的「髒資料」才是真正的核心工程。原始 CSV 檔案中充斥著大量橫線分隔列與不可見的隱形空格，這類格式問題若不透過物理性的 `split()` 切分與 `strip()` 清洗，會直接導致模型計算失效。這讓我意識到，在金融科技的實務應用中，數據預處理往往佔據了開發時程的七成以上，是模型能否發揮價值的關鍵基石。

此外，處理 2021 全年度 244 個交易日的數據規模與處理單一檔案完全不同，這讓我學會了如何設計更具容錯能力的系統架構。最初過度依賴 `pd.read_csv` 的簡便做法，但在面對格式不一的原始資料時頻繁報錯。透過與 AI 的多次迭代討論，我開發出以「智慧錨點」為核心的掃描邏輯，不再死板地指定欄位索引，而是動態偵測買權標記來精準定位數據，這種具備強健性的邏輯設計是確保長達一年份的 Tick 資料能全自動解析的關鍵。

最後，這次實作也重新定義了我對 AI 協作的看法。我發現 AI 雖然能極大化開發效能，但產出的品質完全取決於使用者的「財務領域知識」。若我沒有清楚的金融邏輯，例如設定成交量大於 30 以篩選高流動性資料、判定正確的合約月份，AI 就無法產出具備實務意義的指令。這種「需求提出、錯誤回饋、邏輯優化」的循環過程，不僅解決了原本需數小時手動對照的繁瑣工作，更讓我學會了如何運用「流式處理」思想來優化全年度數據的分析效能。